

## LN19. 集成方法: Boosting 算法

何 琨

计算机学院数据挖掘与机器学习实验室

华中科技大学 Hopcroft 计算科学研究中心

邮箱: [brooklet60@hust.edu.cn](mailto:brooklet60@hust.edu.cn)

2026 年 4 月



- 1 Boosting 概述
- 2 Boosting 解决方案
- 3 搜索弱学习器
- 4 通用 Boosting 算法和提升树
- 5 案例研究 #1: 梯度提升回归树 (GBRT)
- 6 案例研究 #2: AdaBoost
  - 寻找最佳弱学习器
  - 寻找步长  $\alpha$
  - 重归一化
  - AdaBoost 算法
- 7 进一步的分析
- 8 Adaboost 举例
- 9 总结

- 1 Boosting 概述
- 2 Boosting 解决方案
- 3 搜索弱学习器
- 4 通用 Boosting 算法和提升树
- 5 案例研究 #1: 梯度提升回归树 (GBRT)
- 6 案例研究 #2: AdaBoost
  - 寻找最佳弱学习器
  - 寻找步长  $\alpha$
  - 重归一化
  - AdaBoost 算法
- 7 进一步的分析
- 8 Adaboost 举例
- 9 总结

- **基本思路**：对于一个复杂的任务，将多个专家的判断进行适当综合，得出的最终的判断，要比单独任何一个专家的判断好。
- **适用场景**：假设空间  $\mathcal{H}$ ，其分类器集合具有较大的偏差，训练误差较高（例如，深度非常有限的 CART 树，4-6 层）。
- **强可学习和弱可学习**：由 Kearns 和 Valiant 提出。

在概率近似正确 (PAC) 学习的框架中，一个概念（类），如果存在一个多项式的学习算法能够学习它并且准确率很高，则称这个概念为强可学习的。

一个概念（类），如果存在一个多项式的学习算法能够学习它，学习的正确率仅比随机猜测略好，则称这个概念为弱可学习的。

- **著名问题**：在 1988 年的一个机器学习项目中，Kearns 提出了一个著名的问题：能否将弱学习器（H）组合起来，生成一个具有低偏差的强学习器？
- **著名答案**：是的！（Robert Schapire 在 1990 年回答）即强可学习与弱可学习在 PAC 学习的框架下是等价的。
- **中国谚语**：“三个臭皮匠，顶个诸葛亮。”
- **动机**：如果已经有了一些弱学习算法，如何将它们提升（Boosting）为强学习算法？  
==》改变训练数据的概率分布
- **要研究的问题**：
  - ① 在每一轮如何改变训练数据的权值或概率分布？==》e.g., 放大错误样本的权值，减少正确样本的权值
  - ② 如何将弱分类器组合成一个强分类器？==》e.g., 加权多数投票

- 1 Boosting 概述
- 2 Boosting 解决方案
- 3 搜索弱学习器
- 4 通用 Boosting 算法和提升树
- 5 案例研究 #1: 梯度提升回归树 (GBRT)
- 6 案例研究 #2: AdaBoost
  - 寻找最佳弱学习器
  - 寻找步长  $\alpha$
  - 重归一化
  - AdaBoost 算法
- 7 进一步的分析
- 8 Adaboost 举例
- 9 总结

- 创建集成分类器：

$$H_T(\vec{x}) = \sum_{t=1}^T \alpha_t h_t(\vec{x})$$

该集成分类器是以迭代的方式构建的。

在第  $t$  轮迭代中，将分类器  $\alpha_t h_t(\vec{x})$  添加到集成中。

在测试时，评估所有分类器并返回加权和。

- 构建这样一个集成的过程与梯度下降非常相似。  
然而，与在每次迭代中更新模型参数不同，我们向集成中添加的是函数。

设  $\ell$  表示一个（凸且可微的）损失函数。可将其写为：

$$\ell(H) = \frac{1}{n} \sum_{i=1}^n \ell(H(\mathbf{x}_i), y_i)$$

- 设有一个（凸且可微的）损失函数：

$$\ell(H) = \frac{1}{n} \sum_{i=1}^n \ell(H(\mathbf{x}_i), y_i)$$

- 假设已完成了第  $t$  轮迭代，且已有了一个集成分类器  $H_t(\vec{x})$ 。  
在第  $t+1$  轮迭代中，想要向集成中添加另一个弱学习器  $h_{t+1}$ 。  
为此，我们寻找使损失最小的弱学习器：

$$h_{t+1} = \arg \min_{h \in \mathbb{H}} \ell(H_t + \alpha h)$$

- 一旦找到  $h_{t+1}$ ，就将其添加到集成模型中，即：

$$H_{t+1} := H_t + \alpha h$$



- 1 Boosting 概述
- 2 Boosting 解决方案
- 3 搜索弱学习器**
- 4 通用 Boosting 算法和提升树
- 5 案例研究 #1: 梯度提升回归树 (GBRT)
- 6 案例研究 #2: AdaBoost
  - 寻找最佳弱学习器
  - 寻找步长  $\alpha$
  - 重归一化
  - AdaBoost 算法
- 7 进一步的分析
- 8 Adaboost 举例
- 9 总结

# 如何找到弱学习器

**假设：弱学习器的存在性：**

对于加权的数据集  $\mathbf{D}$  和权重  $w_1, \dots, w_n$ , 其中  $w_i > 0$ ,  $\sum_{i=1}^n w_i = 1$ , 一定能找到  $h \in H$  使其加权训练误差小于 0.5。

问题：如何找到这样的  $h \in \mathbb{H}$ ？答案是使用函数空间中的梯度下降。

在函数空间中，内积可以定义为：

$$\langle h, g \rangle = \int h(x)g(x)dx$$

由于只有训练集，将其定义为：

$$\langle h, g \rangle = \sum_{i=1}^n h(\mathbf{x}_i)g(\mathbf{x}_i)$$

## 函数空间中的梯度下降

给定假设空间  $H$ ，想要找到步长  $\alpha$  和（弱学习器） $h$  以最小化损失  $\ell(H + \alpha h)$ 。  
对  $\ell(H + \alpha h)$  使用泰勒近似：

$$\ell(H + \alpha h) \approx \ell(H) + \alpha \langle \nabla \ell(H), h \rangle.$$

这个近似（将  $\ell$  视为线性函数）仅在  $\ell(H)$  周围的小区域内成立，即当  $\alpha$  很小时。

因此，将  $\alpha$  固定为一个小常数（如  $\alpha \approx 0.1$ ）。在固定步长  $\alpha$  的情况下，可以使用上述近似来找到一个近似最优的  $h$ ：

$$\arg \min_{h \in \mathbb{H}} \ell(H + \alpha h) \approx \arg \min_{h \in \mathbb{H}} \langle \nabla \ell(H), h \rangle = \arg \min_{h \in \mathbb{H}} \sum_{i=1}^n \frac{\partial \ell}{\partial [H(\mathbf{x}_i)]} h(\mathbf{x}_i)$$

我们可以将  $\ell(H)$  写为  $\sum_{i=1}^n \ell(H(\mathbf{x}_i)) = \ell(H(\mathbf{x}_1), \dots, H(\mathbf{x}_n))$ （每个预测都是损失函数的输入）：

$$\frac{\partial \ell}{\partial H}(\mathbf{x}_i) = \frac{\partial \ell}{\partial [H(\mathbf{x}_i)]}$$

## 函数空间中的梯度下降

$$\arg \min_{h \in \mathbb{H}} \ell(H + \alpha h) \approx \arg \min_{h \in \mathbb{H}} \langle \nabla \ell(H), h \rangle = \arg \min_{h \in \mathbb{H}} \sum_{i=1}^n \frac{\partial \ell}{\partial [H(\mathbf{x}_i)]} h(\mathbf{x}_i)$$

因此，只要有一个算法  $\mathbb{A}$  来解决

$$h_{t+1} = \arg \min_{h \in \mathbb{H}} \sum_{i=1}^n \underbrace{\frac{\partial \ell}{\partial [H(\mathbf{x}_i)]}}_{r_i} h(\mathbf{x}_i)$$

就可以进行 boosting(提升)。

我们需要一个函数  $\mathbb{A}(\{(\mathbf{x}_1, r_1), \dots, (\mathbf{x}_n, r_n)\}) = \arg \min_{h \in \mathbb{H}} \sum_{i=1}^n r_i h(\mathbf{x}_i)$ 。

这个  $h$  不需要非常优秀。只要  $\sum_{i=1}^n r_i h(\mathbf{x}_i) < 0$ ，我们就可以取得进步。

- 1 Boosting 概述
- 2 Boosting 解决方案
- 3 搜索弱学习器
- 4 通用 Boosting 算法和提升树**
- 5 案例研究 #1: 梯度提升回归树 (GBRT)
- 6 案例研究 #2: AdaBoost
  - 寻找最佳弱学习器
  - 寻找步长  $\alpha$
  - 重归一化
  - AdaBoost 算法
- 7 进一步的分析
- 8 Adaboost 举例
- 9 总结

# 通用 Boosting 算法 (AnyBoost)

---

## 算法 AnyBoost 伪代码

---

**Input:** 损失函数  $\ell$ , 步长  $\alpha$ , 训练样本集  $\{(\mathbf{x}_i, y_i)\}$ , 算法  $\mathbb{A}$

**Output:**  $H_T$

$H_0 \leftarrow 0$

**for**  $t = 0$  **to**  $T - 1$  **do**

    计算梯度:  $\forall i, r_i \leftarrow \frac{\partial \ell((H_t(\mathbf{x}_1), y_1), \dots, (H_t(\mathbf{x}_n), y_n))}{\partial H(\mathbf{x}_i)}$

    选择弱学习器:  $h_{t+1} \leftarrow \mathbb{A}(\{(\mathbf{x}_1, r_1), \dots, (\mathbf{x}_n, r_n)\}) = \arg \min_{h \in \mathbb{H}} \sum_{i=1}^n r_i h(\mathbf{x}_i)$

**if**  $\sum_{i=1}^n r_i h_{t+1}(\mathbf{x}_i) < 0$  **then**

        更新模型:  $H_{t+1} \leftarrow H_t + \alpha_{t+1} h_{t+1}$

**else**

**return**  $H_t$

**end if**

**end for**

**return**  $H_T$

---

- Boosting 方法实际上采用加法模型（即基函数的线性组合）与前向分步算法。
- 以决策树为基函数的 Boosting 方法称为提升树，可以处理分类（对应二叉分类树）和回归问题（对应二叉回归树）。
- 提升树 (Boosting tree) 采用前向分步，每一步通过经验风险最小化确定下一棵决策树的参数。
- 针对不同问题的提升树，主要是选择的损失函数的不同。包括使用平方损失的回归问题，使用指数损失函数的分类问题，以及用一般损失函数的一般决策问题。
- 对于二分类问题，提升树算法只需将 AdaBoost 中的基本分类器限制为二类分类树即可，即可以理解为 AdaBoost 的一种特例。

- 1 Boosting 概述
- 2 Boosting 解决方案
- 3 搜索弱学习器
- 4 通用 Boosting 算法和提升树
- 5 案例研究 #1: 梯度提升回归树 (GBRT)**
- 6 案例研究 #2: AdaBoost
  - 寻找最佳弱学习器
  - 寻找步长  $\alpha$
  - 重归一化
  - AdaBoost 算法
- 7 进一步的分析
- 8 Adaboost 举例
- 9 总结



## 案例研究 #1: 梯度提升回归树 (GBRT)

梯度提升回归树 (Gradient Boosted Regression Tree, GBRT):

- **应用场景:** 分类 ( $y_i \in \{+1, -1\}$ ) 或 (甚至是多维的) 回归 ( $y_i \in \mathcal{R}^k$ )
- **弱学习器:**  $h \in \mathbb{H}$  是回归器,  $h(\mathbf{x}) \in \mathcal{R}, \forall \mathbf{x}$ , 通常是固定深度 (如深度 = 4) 的回归树。
- **步长:**  $\alpha$  固定为一个小常数 (超参数)。
- **损失函数:** 任何可微的凸损失函数, 该损失函数可以分解为样本的损失之和  $\mathcal{L}(H) = \sum_{i=1}^n \ell(H(\mathbf{x}_i))$ 。

为了使用回归树进行梯度提升, 必须能够找到一棵树  $h()$ , 使得:

$$h = \arg \min_{h \in \mathbb{H}} \sum_{i=1}^n r_i h(\mathbf{x}_i)$$

其中  $r_i = \frac{\partial \ell}{\partial H(\mathbf{x}_i)}$ 。

# 案例研究 1: 梯度提升回归树 (GBRT)

目标: 找到一棵树  $h()$ , 使得:  $h = \arg \min_{h \in \mathbb{H}} \sum_{i=1}^n r_i h(\mathbf{x}_i)$ , 其中  $r_i = \frac{\partial \ell}{\partial H(\mathbf{x}_i)}$ 。

我们做出以下假设:

- ①  $\sum_{i=1}^n h^2(\mathbf{x}_i)$  是一个常数。这很容易做到 (通过对预测值进行归一化), 并且很重要, 因为总是可以通过缩放  $h$  来减小  $\sum_{i=1}^n h(\mathbf{x}_i) r_i$ 。

通过将  $\sum_{i=1}^n h^2(\mathbf{x}_i)$  固定为一个常数, 我们实际上是将向量  $h$  限制在一个圆上, 只关心它的方向而不关心它的长度。

- ② CART 树对否定运算是闭合的, 即  $\forall h \in \mathbb{H} \Rightarrow \exists -h \in \mathbb{H}$ 。(这通常是正确的。)
- ③ 我们可以定义负梯度为  $t_i = -r_i$ 。

## 案例研究 1: 梯度提升回归树 (GBRT)

将负梯度定义为  $t_i = -r_i$ 。

$$\begin{aligned} & \arg \min_{h \in \mathbb{H}} \sum_{i=1}^n r_i h(\mathbf{x}_i) \quad (\text{这是原始的 AnyBoost 公式}) \\ &= \arg \min_{h \in \mathbb{H}} -2 \sum_{i=1}^n t_i h(\mathbf{x}_i) \quad (\text{用 } t_i \text{ 替换 } -r_i \text{ 并乘以 } 2, \text{ 这是一个常数}) \\ &= \arg \min_{h \in \mathbb{H}} \sum_{i=1}^n \underbrace{t_i^2}_{\text{常数}} - 2t_i h(\mathbf{x}_i) + \underbrace{(h(\mathbf{x}_i))^2}_{\text{常数}} \quad (\text{添加常数 } \sum_i t_i^2 + h(\mathbf{x}_i)^2) \\ &= \arg \min_{h \in \mathbb{H}} \sum_{i=1}^n (h(\mathbf{x}_i) - t_i)^2 \end{aligned}$$

换句话说, 我们可以使用传统的回归树, 并将值  $t_i$  作为每个  $\mathbf{x}_i$  的标签输入。  
每次迭代时, 为不同的“标签”  $t_1, \dots, t_n$  构建一棵新树。

## 案例研究 1: 梯度提升回归树 (GBRT)

由上一页, 得:

$$\arg \min_{h \in \mathbb{H}} \sum_{i=1}^n r_i h(\mathbf{x}_i) = \arg \min_{h \in \mathbb{H}} \sum_{i=1}^n (h(\mathbf{x}_i) - t_i)^2$$

当损失函数  $\ell$  是均方损失, 即:

$$\ell(H) = \frac{1}{2} \sum_{i=1}^n (H(\mathbf{x}_i) - y_i)^2$$

那么很容易证明:

$$t_i = -\frac{\partial \ell}{\partial H(\mathbf{x}_i)} = y_i - H(\mathbf{x}_i)$$

可看出,  $t_i$  的值就是残差, 即  $t_i$  是从  $\mathbb{H}$  指向  $y$  的向量。

重要的是, 你可以使用任何其他可微的凸损失函数  $\ell$ , 下一个弱学习器  $h()$  的解将始终是使均方损失最小化的回归树。

## 梯度提升回归树 (GBRT)

---

### 算法 GBRT 伪代码

---

**Input:** 损失函数  $\ell$ , 步长  $\alpha$ , 训练样本集  $\{(\mathbf{x}_i, y_i)\}$ , 算法  $\mathbb{A}$

**Output:**  $H$

$H \leftarrow 0$

**for**  $t = 1$  **to**  $T$  **do**

$\forall i, t_i \leftarrow y_i - H(\mathbf{x}_i)$

$h \leftarrow \arg \min_{h \in \mathbb{H}} \sum_{i=1}^n (h(\mathbf{x}_i) - t_i)^2$

$H \leftarrow H + \alpha h$

**end for**

**return**  $H$

---

# 梯度提升树总结

梯度提升算法首先给定一个损失函数 (目标函数), 它的定义域是所有可行的弱函数集合 (基函数)。提升算法通过迭代地选择一个负梯度方向上的基函数来逐渐逼近局部极小值。

梯度提升包括两类:

- 梯度提升回归树 (GBRT): 处理回归问题
- 梯度提升决策树 (GBDT): 处理决策问题

即梯度提升树为多个决策树/回归树分类器的组合。

- ① 首先要确定损失函数与初始化决策树分类器  $f_0(x) = 0$
- ② 第  $t$  提升树的参数是使得当前第  $t$  步梯度提升模型的损失函数最小化
- ③ 第  $t$  步的参数是通过梯度下降法求得的残差间接得到的
- ④ 最小化损失函数的过程是根据提升树中决策树的数目而不断重复的过程 (相当于对损失函数做了多次优化, 多次求解极小值)
- ⑤ 对于残差的理解: 前一步提升树所遗留下来的损失值 ( $t-1$  步的损失函数的最小值), 然后通过当前步的决策树分类器做进一步的减小

- 1 Boosting 概述
- 2 Boosting 解决方案
- 3 搜索弱学习器
- 4 通用 Boosting 算法和提升树
- 5 案例研究 #1: 梯度提升回归树 (GBRT)
- 6 案例研究 #2: AdaBoost
  - 寻找最佳弱学习器
  - 寻找步长  $\alpha$
  - 重归一化
  - AdaBoost 算法
- 7 进一步的分析
- 8 Adaboost 举例
- 9 总结

AdaBoost 任务设置:

- 设置: 分类任务 ( $y_i \in \{+1, -1\}$ )
- 弱学习器:  $h \in \mathbb{H}$  是二元分类器,  $h(\mathbf{x}_i) \in \{-1, +1\}, \forall \mathbf{x}$
- 步长: 我们执行线搜索 (line-search) 以获得最佳步长  $\alpha$
- 损失函数: 指数损失  $\ell(H) = \sum_{i=1}^n e^{-y_i H(\mathbf{x}_i)}$



损失函数：指数损失  $\ell(H) = \sum_{i=1}^n e^{-y_i H(\mathbf{x}_i)}$

首先，计算梯度：

$$r_i = \frac{\partial \ell}{\partial H(\mathbf{x}_i)} = ?$$

损失函数：指数损失  $\ell(H) = \sum_{i=1}^n e^{-y_i H(\mathbf{x}_i)}$

首先，计算梯度：

$$r_i = \frac{\partial \ell}{\partial H(\mathbf{x}_i)} = -y_i e^{-y_i H(\mathbf{x}_i)}$$

为了方便表示，定义：

$$w_i = \frac{1}{Z} e^{-y_i H(\mathbf{x}_i)}$$

其中  $Z = \sum_{i=1}^n e^{-y_i H(\mathbf{x}_i)}$  是一个归一化因子，使得  $\sum_{i=1}^n w_i = 1$

**损失函数：**指数损失  $\ell(H) = \sum_{i=1}^n e^{-y_i H(\mathbf{x}_i)}$

首先，计算梯度：

$$r_i = \frac{\partial \ell}{\partial H(\mathbf{x}_i)} = -y_i e^{-y_i H(\mathbf{x}_i)}$$

为了方便表示，定义：

$$w_i = \frac{1}{Z} e^{-y_i H(\mathbf{x}_i)}$$

其中  $Z = \sum_{i=1}^n e^{-y_i H(\mathbf{x}_i)}$  是一个归一化因子，使得  $\sum_{i=1}^n w_i = 1$ 。

注意到归一化常数  $Z$  与损失函数相同。每个权重  $w_i$  因此有一个很好的解释：它是训练点  $(\mathbf{x}_i, y_i)$  对总体损失的相对贡献。

## 寻找最佳弱学习器

为了找到下一个最佳弱学习器，我们需要解决优化问题（在以下内容中，将利用  $h(\mathbf{x}_i) \in \{+1, -1\}$  的事实）：

$$\begin{aligned}h(\mathbf{x}_i) &= \arg \min_{h \in \mathbb{H}} \sum_{i=1}^n r_i h(\mathbf{x}_i) \quad \left( \text{代入: } r_i = -y_i e^{-H(\mathbf{x}_i)y_i} \right) \\&= \arg \min_{h \in \mathbb{H}} - \sum_{i=1}^n y_i e^{-H(\mathbf{x}_i)y_i} h(\mathbf{x}_i) \quad \left( \text{代入: } w_i = \frac{1}{Z} e^{-H(\mathbf{x}_i)y_i} \right) \\&= \arg \min_{h \in \mathbb{H}} - \sum_{i=1}^n w_i y_i h(\mathbf{x}_i) \quad (y_i h(\mathbf{x}_i) \in \{+1, -1\} \text{ 且 } h(\mathbf{x}_i)y_i = 1 \Leftrightarrow h(\mathbf{x}_i) = y_i) \\&= \arg \min_{h \in \mathbb{H}} \sum_{i: h(\mathbf{x}_i) \neq y_i} w_i - \sum_{i: h(\mathbf{x}_i) = y_i} w_i \quad \left( \text{因为 } \sum_{i: h(\mathbf{x}_i) = y_i} w_i = 1 - \sum_{i: h(\mathbf{x}_i) \neq y_i} w_i \right) \\&= \arg \min_{h \in \mathbb{H}} \sum_{i: h(\mathbf{x}_i) \neq y_i} w_i \quad (\text{这是加权分类误差})\end{aligned}$$

## 加权分类误差

$$\arg \min_{h \in \mathbb{H}} \sum_{i: h(\mathbf{x}_i) \neq y_i} w_i$$

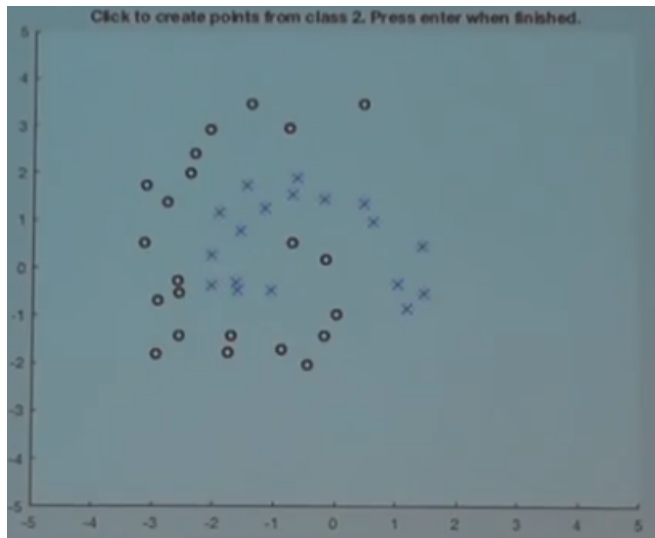
我们称这个加权分类误差为：

$$\epsilon = \sum_{i: h(\mathbf{x}_i) \neq y_i} w_i$$

对于 AdaBoost，我们只需要一个分类器，它能够接受训练数据和训练集上的分布（即，所有训练样本的归一化权重  $w_i$ ），并返回一个分类器  $h \in \mathbb{H}$ ，该分类器减少了这些训练样本的加权分类误差。

它不需要表现得非常好，只要加权训练误差小于  $\epsilon < 0.5$ ，内积  $\sum_i r_i h(\mathbf{x}_i)$  就会是负数。

## 运行示例



图：训练点集

## 运行示例

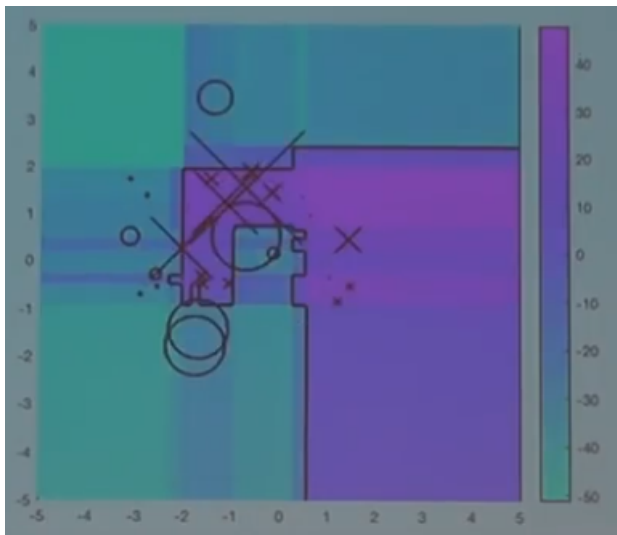
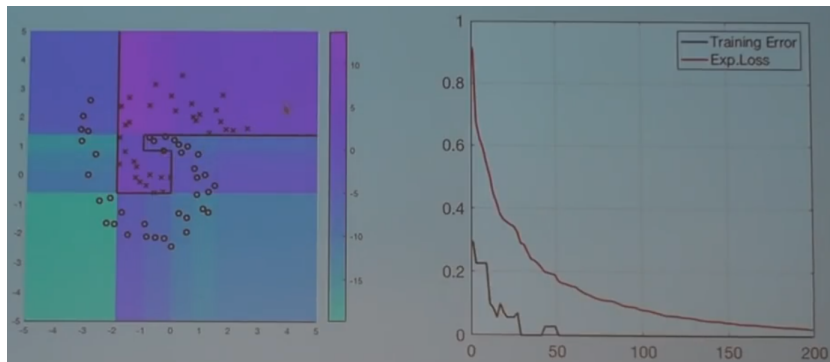


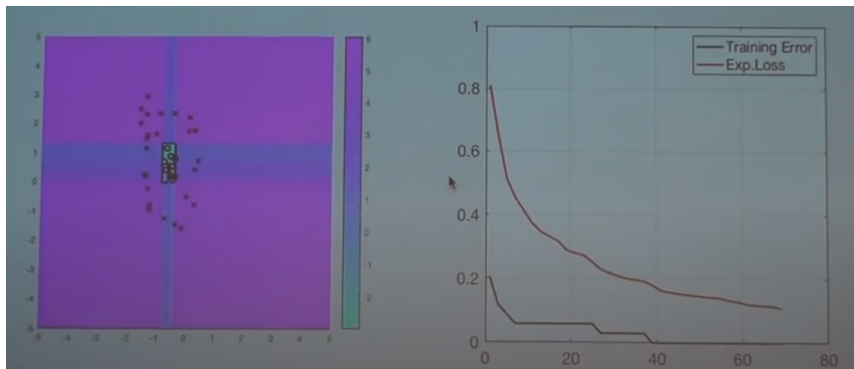
图: 对训练点加权

# 运行示例





# 运行示例



## 寻找步长 $\alpha$

在前面的例子中 (GBRT), 我们将步长  $\alpha$  设置为一个小的常数。

事实证明, 在 AdaBoost 设置中, 可以在每次“梯度”迭代时以闭式形式找到最优步长 (即, 使  $\ell$  最小化的步长)。

当给定  $\ell, H, h$  时, 需要解决以下优化问题:

$$\begin{aligned}\alpha &= \arg \min_{\alpha} \ell(H + \alpha h) \\ &= \arg \min_{\alpha} \sum_{i=1}^n e^{-y_i[H(\mathbf{x}_i) + \alpha h(\mathbf{x}_i)]}\end{aligned}$$

对  $\alpha$  求导并令其等于零:

$$\sum_{i=1}^n y_i h(\mathbf{x}_i) e^{-y_i[H(\mathbf{x}_i) + \alpha h(\mathbf{x}_i)]} = 0 \quad (y_i h(\mathbf{x}_i) \in \{+1, -1\})$$

## 寻找步长 $\alpha$

对  $\alpha$  求导并令其等于零：

$$\begin{aligned} \sum_{i=1}^n y_i h(\mathbf{x}_i) e^{-y_i [H(\mathbf{x}_i) + \alpha h(\mathbf{x}_i)]} &= 0 \quad (y_i h(\mathbf{x}_i) \in \{+1, -1\}) \\ - \sum_{i: h(\mathbf{x}_i) y_i = -1} e^{-(y_i H(\mathbf{x}_i) + \alpha)} + \sum_{i: h(\mathbf{x}_i) y_i = 1} e^{-(y_i H(\mathbf{x}_i) - \alpha)} &= 0 \quad \left( w_i = \frac{1}{Z} e^{-y_i H(\mathbf{x}_i)} \right) \\ - \sum_{i: h(\mathbf{x}_i) y_i = -1} w_i e^{-\alpha} + \sum_{i: h(\mathbf{x}_i) y_i = 1} w_i e^{\alpha} &= 0 \quad \left( \epsilon = \sum_{i: h(\mathbf{x}_i) y_i = -1} w_i \right) \\ -(1 - \epsilon) e^{-\alpha} + \epsilon e^{\alpha} &= 0 \\ e^{2\alpha} &= \frac{1 - \epsilon}{\epsilon} \\ \alpha &= \frac{1}{2} \ln \frac{1 - \epsilon}{\epsilon} \end{aligned}$$

在 AdaBoost 中，我们能够以如此简单的闭式形式找到最优步长是不常见的。  
其结果就是，AdaBoost 收敛得非常快。

# 重归一化

在迭代一步后，即  $H_{t+1} = H_t + \alpha h$ ，需要重新计算所有权重并进行重新归一化。

不过，可以直接证明，未归一化的权重  $\hat{w}_i$  会更新为：

$$\hat{w}_i \leftarrow \hat{w}_i * e^{-\alpha h(\mathbf{x}_i)y_i}$$

并且归一化因子  $Z$  会更新为：

$$Z \leftarrow Z * 2\sqrt{\epsilon(1-\epsilon)}.$$

将这两者结合，我们得到以下乘法更新规则：

$$w_i \leftarrow w_i \frac{e^{-\alpha h(\mathbf{x}_i)y_i}}{2\sqrt{\epsilon(1-\epsilon)}}.$$

## 算法 AdaBoost 伪代码

**Input:** 损失函数  $\ell$ , 步长  $\alpha$ , 训练样本集  $\{(\mathbf{x}_i, y_i)\}$ , 算法  $\mathbb{A}$

**Output:**  $H_T$

$H_0 \leftarrow 0$

$\forall i: w_i \leftarrow \frac{1}{n}$

**for**  $t = 0$  **to**  $T - 1$  **do**

$h \leftarrow \arg \min_h \sum_{i: h(\mathbf{x}_i) \neq y_i} w_i \quad [h = \mathbb{A}((w_1, \mathbf{x}_1, y_1), \dots, (w_n, \mathbf{x}_n, y_n))]$

$\epsilon \leftarrow \sum_{i: h(\mathbf{x}_i) \neq y_i} w_i$

**if**  $\epsilon < \frac{1}{2}$  **then**

$\alpha \leftarrow \frac{1}{2} \ln \left( \frac{1-\epsilon}{\epsilon} \right)$

$H_{t+1} \leftarrow H_t + \alpha h$

$\forall i: w_i \leftarrow \frac{w_i e^{-\alpha h(\mathbf{x}_i) y_i}}{2 \sqrt{\epsilon(1-\epsilon)}}$

**else**

**return**  $H_t$

**end if**

**end for**

**return**  $H_T$

- 只要  $H$  是“否定”闭合的（这意味着对于每个  $h \in H$ ，必须有  $-h \in H$ ），误差  $\epsilon$  就不会大于  $\frac{1}{2}$ 。  
原因很简单，如果  $h$  的误差是  $\epsilon$ ，那么  $-h$  的误差必须是  $1 - \epsilon$ 。因此，你可以将  $h$  翻转为  $-h$  以获得一个误差更小的分类器。  
由于  $h$  是通过最小化误差找到的，从而引出矛盾。
- 内循环可以在误差  $\epsilon = \frac{1}{2}$  时终止，在大多数情况下，它会随时间收敛到  $\frac{1}{2}$ 。  
在这种情况下，最新的弱学习器  $h$  只会与抛硬币一样，误差为  $\epsilon = \frac{1}{2}$ ，因此无法从集成中获益（因此，Boosting 终止）。  
还需要注意的是，如果  $\epsilon = \frac{1}{2}$ ，步长  $\alpha$  将为零。

# 目录

- 1 Boosting 概述
- 2 Boosting 解决方案
- 3 搜索弱学习器
- 4 通用 Boosting 算法和提升树
- 5 案例研究 #1: 梯度提升回归树 (GBRT)
- 6 案例研究 #2: AdaBoost
  - 寻找最佳弱学习器
  - 寻找步长  $\alpha$
  - 重归一化
  - AdaBoost 算法
- 7 进一步的分析**
- 8 Adaboost 举例
- 9 总结

## 进一步的分析

让我们逐一检查这些更新：

- \*\* 权重更新 \*\*：

$$\hat{w}_i \leftarrow \hat{w}_i * e^{-\alpha h(\mathbf{x}_i)y_i}$$

由于  $h(\mathbf{x}_i)y_i$  要么是  $+1$ （如果该弱学习器正确分类），要么是  $-1$ （否则），因此权重更新会将权重  $\hat{w}_i$  乘以一个因子  $e^\alpha > 1$ （如果分类错误，即增加权重），或者乘以一个因子  $e^{-\alpha} < 1$ （如果分类正确，即减少权重）。

- \*\* 归一化更新 \*\*：

$$Z \leftarrow Z * 2\sqrt{\epsilon(1-\epsilon)}$$

之前已经确定归一化因子  $Z$  与损失函数相同。因此，可以利用它来限制  $T$  次迭代后的损失函数：

$$\ell(H) = Z = n \prod_{t=1}^T 2\sqrt{\epsilon_t(1-\epsilon_t)}$$

（因子  $n$  来自初始  $Z_0 = n$ ，当所有权重都是  $\frac{1}{n}$  时。）如果定义  $c = \max_t \epsilon_t$ ，可以得出：

$$\ell(H) \leq n \left[ 2\sqrt{c(1-c)} \right]^T$$



## 进一步分析

函数  $c(1-c)$  在  $c = \frac{1}{2}$  时达到最大值。但我们知道每个  $\epsilon_t < \frac{1}{2}$  (否则算法会终止)。因此  $c(1-c) < \frac{1}{4}$ , 可以将其重写为  $c(1-c) = \frac{1}{4} - \gamma^2$ , 其中  $\gamma$  是某个值。这使我们得到:

$$\ell(H) \leq n(1 - 4\gamma^2)^{\frac{T}{2}}$$

换句话说, 训练损失呈指数级下降!

实际上, 我们甚至可以进一步计算出需要多少次迭代才能使训练误差为零。

注意, 训练损失是训练误差的上界 (定义为  $\sum_{i=1}^n \delta_{H(x_i) \neq y_i}$ , 因为  $\delta_{H(x_i) \neq y_i} < e^{-y_i H(x_i)}$  在所有情况下成立。

我们可以计算出需要的步数, 直到损失小于 1, 这意味着没有一个训练输入被错误分类:

$$n(1 - 4\gamma^2)^{\frac{T}{2}} < 1 \Rightarrow T > \frac{2 \log(n)}{\log\left(\frac{1}{1-4\gamma^2}\right)}$$

这是一个惊人的结果。它表明, 在  $O(\log(n))$  次迭代后, 你的训练误差必然为零。

在实践中, 即使在训练集上不再犯错后, 继续进行 boosting 通常也是有意义的。

- 1 Boosting 概述
- 2 Boosting 解决方案
- 3 搜索弱学习器
- 4 通用 Boosting 算法和提升树
- 5 案例研究 #1: 梯度提升回归树 (GBRT)
- 6 案例研究 #2: AdaBoost
  - 寻找最佳弱学习器
  - 寻找步长  $\alpha$
  - 重归一化
  - AdaBoost 算法
- 7 进一步的分析
- 8 Adaboost 举例
- 9 总结

## Adaboost 举例

**问题：**假设有以下数据，弱分类器由  $x < v$  和  $x > v$  产生，其中阈值  $v$  使该分类器在训练集上分类误差率最低。请用 Adaboost 算法学习一个强分类器。（参考：《统计学习方法》-李航）

|     |   |   |   |    |    |    |   |   |   |    |
|-----|---|---|---|----|----|----|---|---|---|----|
| $x$ | 1 | 2 | 3 | 4  | 5  | 6  | 7 | 8 | 9 | 10 |
| $y$ | 1 | 1 | 1 | -1 | -1 | -1 | 1 | 1 | 1 | -1 |

表：训练数据集

**解：**第一轮：

1)  $H_0 = 0$ , 初始化数据权值分布:  $w_i = 0.1, i = 1 \dots 10$

2) 在当前权值分布的训练数据上，阈值  $v$  取 2.5 时分类误差率最低，分错的比例  $\epsilon$  为 0.3:

$$h_1(x) = \begin{cases} 1, v < 2.5, \\ -1, v > 2.5, \end{cases}$$

3) 计算  $h_1(x)$  的系数:

$$\alpha_1 = 0.5 \ln \frac{1 - \epsilon}{\epsilon} = 0.4236$$

4) 更新  $H$ , 该  $H$  在数据集上有 3 个误分类点:

$$H_1 \leftarrow H_0 + \alpha_1 h_1 = \alpha h_1$$

## Adaboost 举例

|   |   |   |   |    |    |    |   |   |   |    |
|---|---|---|---|----|----|----|---|---|---|----|
| x | 1 | 2 | 3 | 4  | 5  | 6  | 7 | 8 | 9 | 10 |
| y | 1 | 1 | 1 | -1 | -1 | -1 | 1 | 1 | 1 | -1 |

第二轮：1) 更新训练数据的权重分布：由  $w_i \leftarrow \frac{w_i e^{-\alpha h(x_i)y_i}}{2\sqrt{\epsilon(1-\epsilon)}}$ ，归一化因子  $Z$ ：  
 $Z \leftarrow Z * 2\sqrt{\epsilon(1-\epsilon)}$ ：

$$w_i \leftarrow \frac{w_i e^{-\alpha h(x_i)y_i}}{Z_i}$$

可得新的权重： $W_2 = (0.07143, 0.07143, 0.07143, 0.07143, 0.07143,$   
 $0.07143, 0.16667, 0.16667, 0.16667, 0.07143)$

2) 在新的权值分布的数据上，阈值  $v$  取 8.5 时分类误差率最低，误差率  $\epsilon$  为 0.2143：

$$h_1(x) = \begin{cases} 1, v < 8.5, \\ -1, v > 8.5, \end{cases}$$

3) 计算  $h_2(x)$  的系数：

$$\alpha_2 = 0.5 \ln \frac{1-\epsilon}{\epsilon} = 0.6496$$

4) 更新  $H$ ，该  $H$  在数据集上有 3 个误分类点

$$H_2 \leftarrow H_1 + \alpha_2 h_2 = 0.4236h_1 + 0.6496h_2$$

## Adaboost 举例

|     |   |   |   |    |    |    |   |   |   |    |
|-----|---|---|---|----|----|----|---|---|---|----|
| $x$ | 0 | 1 | 2 | 3  | 4  | 5  | 6 | 7 | 8 | 9  |
| $y$ | 1 | 1 | 1 | -1 | -1 | -1 | 1 | 1 | 1 | -1 |

第三轮：1) 更新训练数据的权重分布： $w_i \leftarrow \frac{w_i e^{-\alpha h(x_i)y_i}}{Z_i}$

可得新的权重：

$$W_2 = (0.0455, 0.0455, 0.0455, 0.1667, 0.1667, \\ 0.1667, 0.1060, 0.1060, 0.1060, 0.0455)$$

2) 在新的权值分布的数据上，阈值  $v$  取 5.5 时分类误差率最低，误差率  $\epsilon$  为 0.1820：

$$h_1(x) = \begin{cases} 1, & v > 5.5, \\ -1, & v < 5.5, \end{cases}$$

3) 计算  $h_3(x)$  的系数：

$$\alpha_3 = 0.5 \ln \frac{1 - \epsilon}{\epsilon} = 0.7514$$

4) 更新  $H$ :  $H_2 \leftarrow H_2 + \alpha_3 h_3 = 0.4236h_1 + 0.6496h_2 + 0.7514h_3$

可以验证，该分类器在训练数据集上的误差为 0。最终的分类器为：

$$H = \text{sign}[0.4236h_1 + 0.6496h_2 + 0.7514h_3]$$

# 目录

- 1 Boosting 概述
- 2 Boosting 解决方案
- 3 搜索弱学习器
- 4 通用 Boosting 算法和提升树
- 5 案例研究 #1: 梯度提升回归树 (GBRT)
- 6 案例研究 #2: AdaBoost
  - 寻找最佳弱学习器
  - 寻找步长  $\alpha$
  - 重归一化
  - AdaBoost 算法
- 7 进一步的分析
- 8 Adaboost 举例
- 9 总结

## 总结

- Boosting 是一种将弱分类器转化为强分类器的优秀方法。它定义了一系列算法，包括梯度提升 Gradient Boosting、AdaBoost、LogitBoost 等。
- 梯度提升回归树 (Gradient Boosted Regression Trees, GBRT) 是学习排序 (Learning to Rank) 领域中最受欢迎的算法之一，如在网页搜索引擎中有广泛的应用。

此外，还有一些需要注意的点：

- 步长  $\alpha$  通常被称为收缩因子 (shrinkage)。
- 有些人认为梯度提升算法不属于传统的 Boosting 算法家族，因为它们不能保证训练误差呈指数级下降。这些算法通常被称为分段回归 (stage-wise regression)。
- 受 Breiman 的 Bagging 启发，随机梯度提升对每个弱学习器的训练数据进行子采样。这结合了 Bagging 和 Boosting 的优势。一种变体是对每个弱学习器仅进行无放回的  $n/2$  数据点子采样，以加速训练过程。

其他需要注意的点：

- 提升 (Boosting) 分类器的一个优势是在测试时，如果可以提前确定预测的方向，则可以提前停止计算：

$$H(\mathbf{x}) = \sum_{t=1}^T \alpha_t h_t(\mathbf{x})$$

这在搜索引擎中特别有用，因为通常只有前 10 个搜索结果的排名是重要的。停止对较低排名的搜索结果的评估可以带来巨大的速度提升。

- Viola-Jones 算法也采用了类似的方法来加速图像中的面部检测。其中，算法扫描图像区域以检测可能的面部。  
由于大多数区域和自然图像不包含面部，如果在评估了几个弱学习器后就可以停止评估，将节约大量的时间。  
这些分类器被称为级联，它们在大多数情况（无面部）下花费很少的时间，但在少数情况（有面部）下花费更多的时间。
- AdaBoost 是一种极其强大的算法，它将任何能够在加权训练集上实现低于 0.5 误差的弱学习器转化为强学习器，其训练误差呈指数级下降，并且只需要  $O(\log(n))$  步即可收敛。



## 三个臭皮匠，顶个诸葛亮

三个普通人的智慧合起来，也能抵得上一个天才（诸葛亮）

## 三个臭裨将，顶个诸葛亮。

“裨将”指副将或低级军官。

# The End